

Roteiro Aula Prática



PROGRAMAÇÃO WEB II

ROTEIRO DE AULA PRÁTICA

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB

Unidade: U1_ PROGRAMAÇÃO_EM_JSF

Aula: A2_ COMPONENTES_VISUAIS_DO_JSF

OBJETIVOS

Definição dos objetivos da aula prática:

Definição dos objetivos da aula prática: Ensinar o aluno a criar um projeto web dinâmico utilizando conceitos iniciais de JSF na camada de visão, além de estruturar as partes desse projeto segundo o padrão de arquitetura MVC, tudo isso sendo aplicado à linguagem de programação Java. Após a criação e estruturação dos arquivos, aplicaremos uma folha de estilo no formulário criado.

SOLUÇÃO DIGITAL:

Eclipse e Wildfly

Ide Eclipse ou correlata que possua configurações para construção de classes seguindo o Java EE, com configurações para ter servidor de aplicação e utilizar o framework JSF para a camada de visão. Recomenda-se o uso do servidor wilfly.

1. Acesse o site oficial: <https://eclipseide.org/>
2. Clique em Download:



3. Escolha a versão “Eclipse IDE for Enterprise Java and Web Developers” que compatível com o sistema operacional do seu computador.



4. Após o download, descompacte o já estará pronto para uso.

PROCEDIMENTOS PRÁTICOS E APLICAÇÕES

Procedimento/Atividade nº 1

CONSTRUINDO UM CONJUNTO DE CLASSES COM O MVC E JSF.

Atividade proposta: Criaremos um projeto baseado em Java EE (Projeto de WEB Dinâmico) com organização preconizada pelo MVC, para cadastro de faculdades.

Você deve elaborar, pelo menos, uma classe de modelo chamada Faculdade. Essa classe terá um id (int), nomeCompleto (String) e nomeFantasia (String). Esses dados devem ser obtidos através de um formulário que contenha esses três elementos para ser digitados pelo usuário. Esse formulário deve estar presente na página faculdades.xhtml e ela deve obrigatoriamente utilizar conceitos de JSF que seja compatível com as premissas de utilização de ManagedBeans. A classe Java FaculdadeMB será a responsável por controlar as requisições oriundas da camada de visão e passá-las para a camada modelo. As etapas descritas a seguir foram criadas usando a IDE Eclipse, com o servidor Wildfly.

Procedimentos para a realização da atividade:

Para iniciar a atividade, vamos criar um projeto web dinâmico:

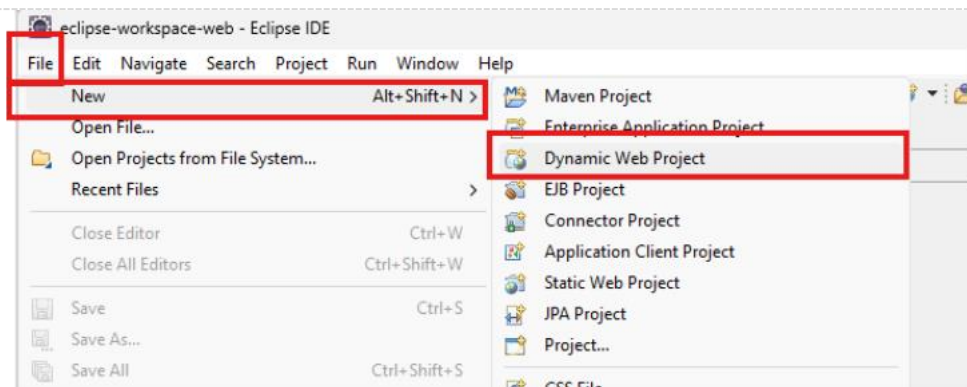


Figura 1 – Criando um Dynamic Web Project

Vamos dar o nome “Faculdade” ao projeto e colocar as configurações conforme a Figura 7. Caso seu “Target runtime” esteja vazio como na Figura 2 então clique em “New Runtime...”, então uma janela abrirá, procure por “WildFly” da mesma forma que a Figura 3, selecione “JBoss AS, WildFly & EAP Server Tools” e clique em next, prossiga com a instalação até a conclusão, e no final, aceite os termos. Atenção, mesmo após o aceite dos termos o download e a instalação ainda podem estar ocorrendo, cheque no canto inferior esquerdo (Figura 4) e logo após a finalização o Eclipse precisará ser reiniciado, então repita o processo de criação do projeto, caso ainda não tenha o “Target runtime” clique novamente em “New Runtime...” e procure por “WildFly” (Figura 5), então selecione “WildFly 10.x Runtime” e clique em next, na próxima clique em “Download and install runtime...”, vamos utilizar a versão, selecione ela e clique em next, prossiga com a instalação até a finalização, agora podemos prosseguir com a configuração do projeto.

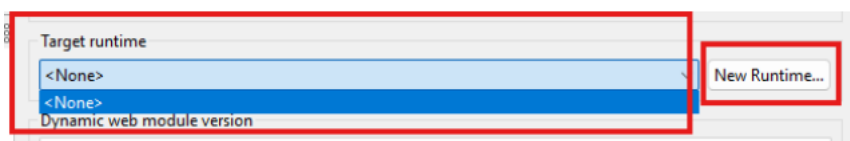


Figura 2 – Configurando runtime

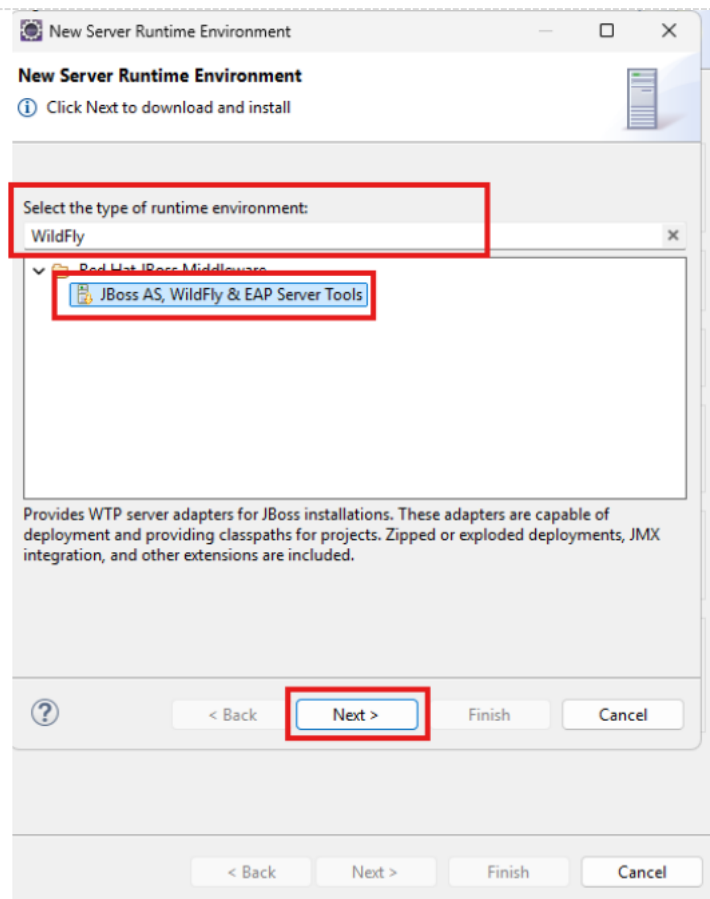


Figura 3 – Instalando WildFly

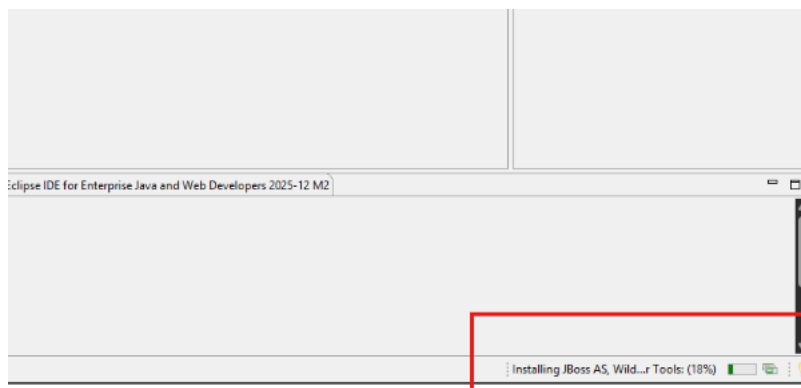


Figura 4 – Processo de download e instalação

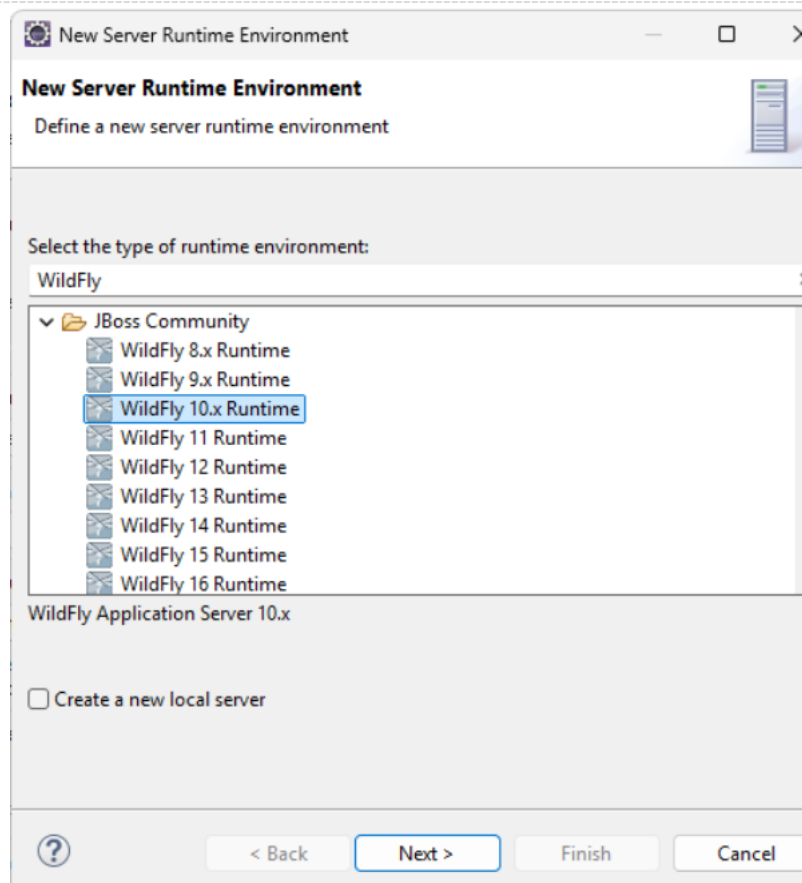


Figura 5 – Seleção Runtime WildFly 10

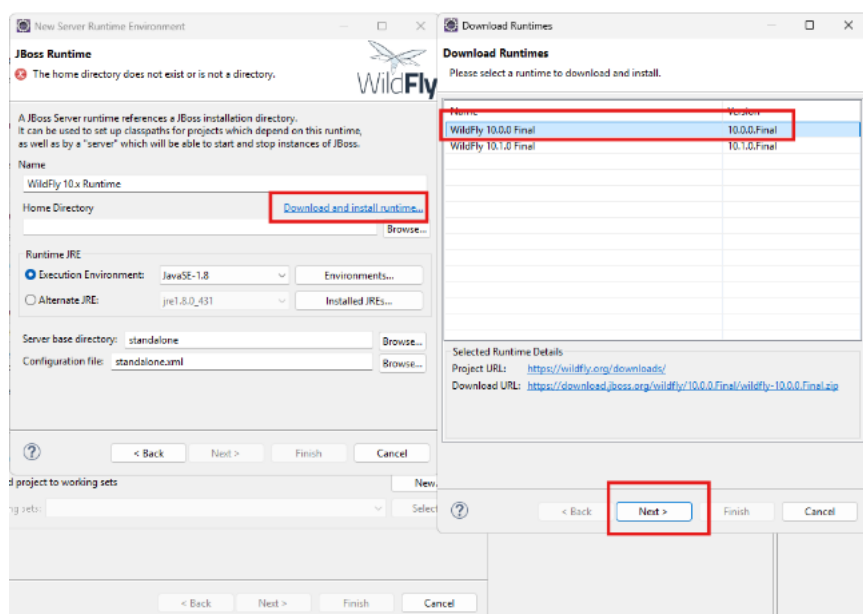


Figura 6 – Instalação WildFly 10

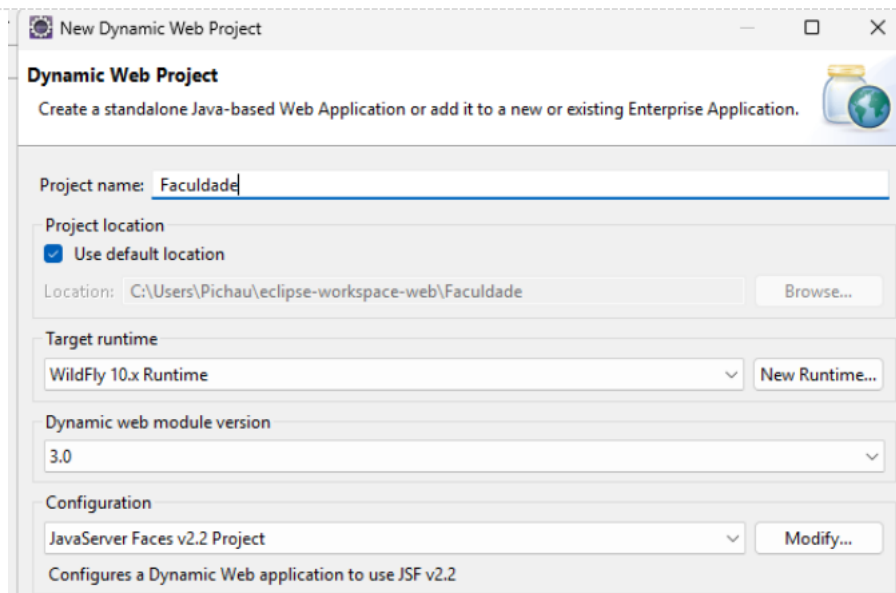


Figura 7 – Configurações para criação do projeto

Para criação do formulário, clique com o botão direito sobre o projeto e escolha New >> HTML File. Dê o nome para o arquivo de faculdades.xhtml e clique em finish. Crie os campos necessários usando os recursos JSF, conforme código na Figura 8.

```

1  <!DOCTYPE html>
2  <html xmlns="http://www.w3.org/1999/xhtml" xmlns:h="http://xmlns.jcp.org/jsf/html">
3  <h: head>
4      <meta charset="UTF-8"/>
5      <title>Cadastro de Faculdade</title>
6      <h:outputStylesheet library="css" name="estilo1.css" />
7  </h:head>
8  <h:body>
9      <h:form>
10         <p class="rotulo">Código:
11             <h:inputText value="#{faculdadeMB.codigo}" class="entrada" />
12         </p>
13         <p class="rotulo">Nome completo:
14             <h:inputText value="#{faculdadeMB.nomeC}" class="entrada" />
15         </p>
16         <p class="rotulo">Nome fantasia:
17             <h:inputText value="#{faculdadeMB.nomeF}" class="entrada" />
18         </p>
19         <p class="botao">
20             <h:commandButton value="Submit" action="faculdades" />
21         </p>
22     </h:form>
23     <p id="resultado">Faculdade cadastrada: <br/>
24         Código: "#{faculdadeMB.codigo}"<br/>
25         Nome: "#{faculdadeMB.nomeC}"<br/>
26         Nome Fantasia: "#{faculdadeMB.nomeF}"<br/>
27     </p>
28 </h:body>
29
30 </html>

```

Figura 8 – Código para criação do formulário faculdade.xhtml o

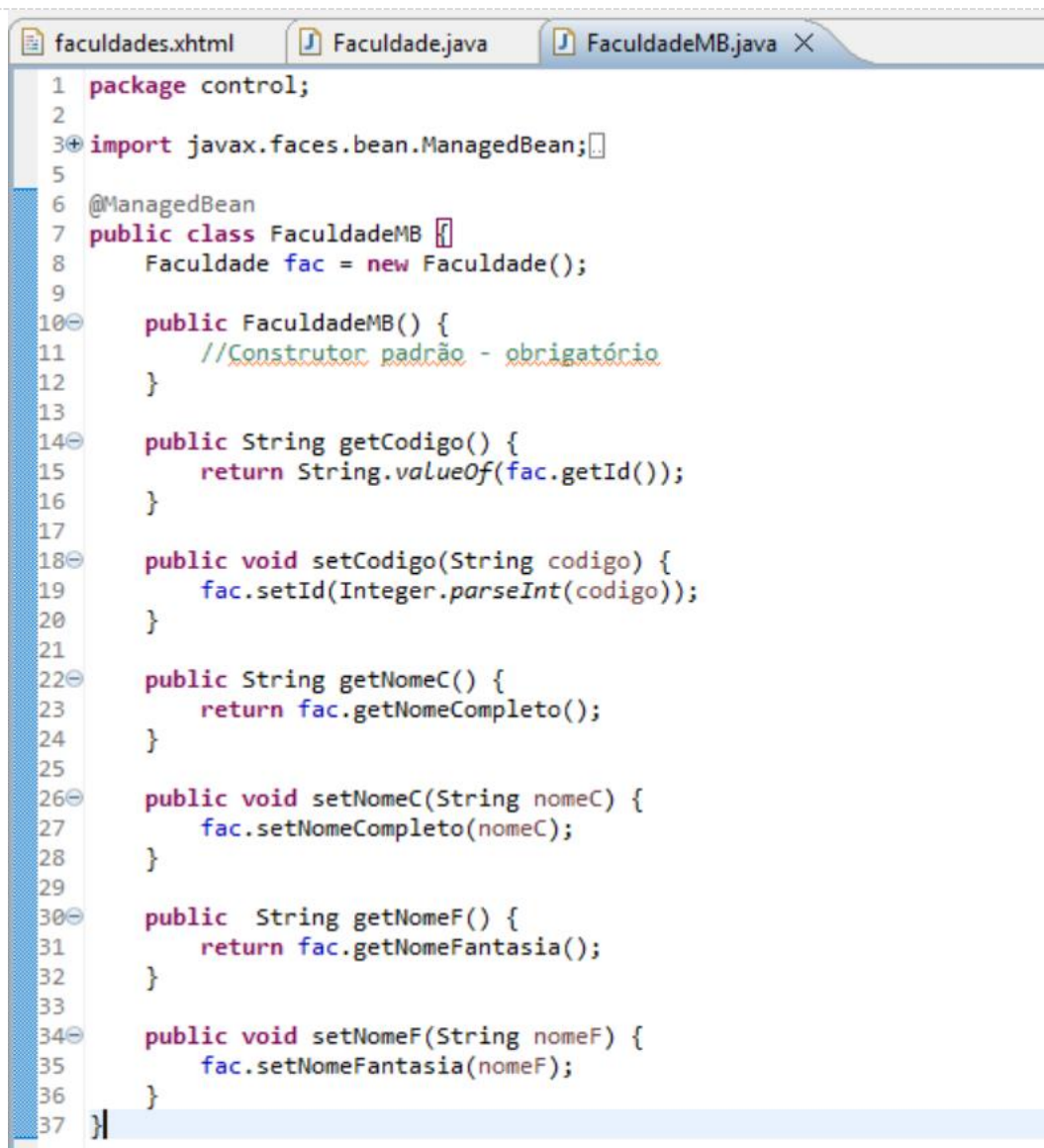
Criada a interface para a tela de cadastro, o próximo passo é criar o pacote “model” e dentro do pacote criar a classe “Faculdade”, com o código da Figura 9.



```
1 package model;
2
3 public class Faculdade{
4
5     private int id;
6     private String nomeCompleto;
7     private String nomeFantasia;
8
9     public int getId() {
10         return id;
11     }
12
13     public void setId(int id) {
14         this.id = id;
15     }
16
17     public String getNomeCompleto() {
18         return nomeCompleto;
19     }
20
21     public void setNomeCompleto(String nomeCompleto) {
22         this.nomeCompleto = nomeCompleto;
23     }
24
25     public String getNomeFantasia() {
26         return nomeFantasia;
27     }
28
29     public void setNomeFantasia(String nomeFantasia) {
30         this.nomeFantasia = nomeFantasia;
31     }
32 }
```

Figura 9 – Classe Faculdade

O próximo passo consiste em criar uma classe especial, uma Managed Bean, para fazer o controle da comunicação entre a camada de visão e a camada modelo. Para isso, no projeto crie um pacote chamado “control” e dentro do pacote uma nova classe Java chamada “FaculdadeMB”, com o código da Figura 10.



```

1 package control;
2
3 import javax.faces.bean.ManagedBean;
4
5
6 @ManagedBean
7 public class FaculdadeMB {
8     Faculdade fac = new Faculdade();
9
10    public FaculdadeMB() {
11        //Construtor padrão - obrigatório
12    }
13
14    public String getCodigo() {
15        return String.valueOf(fac.getId());
16    }
17
18    public void setCodigo(String codigo) {
19        fac.setId(Integer.parseInt(codigo));
20    }
21
22    public String getNomeC() {
23        return fac.getNomeCompleto();
24    }
25
26    public void setNomeC(String nomeC) {
27        fac.setNomeCompleto(nomeC);
28    }
29
30    public String getNomeF() {
31        return fac.getNomeFantasia();
32    }
33
34    public void setNomeF(String nomeF) {
35        fac.setNomeFantasia(nomeF);
36    }
37 }

```

Figura 10 – Classe FaculdadeMB

O próximo passo é configurar o arquivo faces-config.xml, que se encontra dentro da pasta webapp > WEB-INF, clique duas vezes para abrir, caso de primeira não consiga ver o código, no menu inferior dessa janela clique em “source”. Com o código da Figura 11, adicione as tags das linhas 8 a 10.

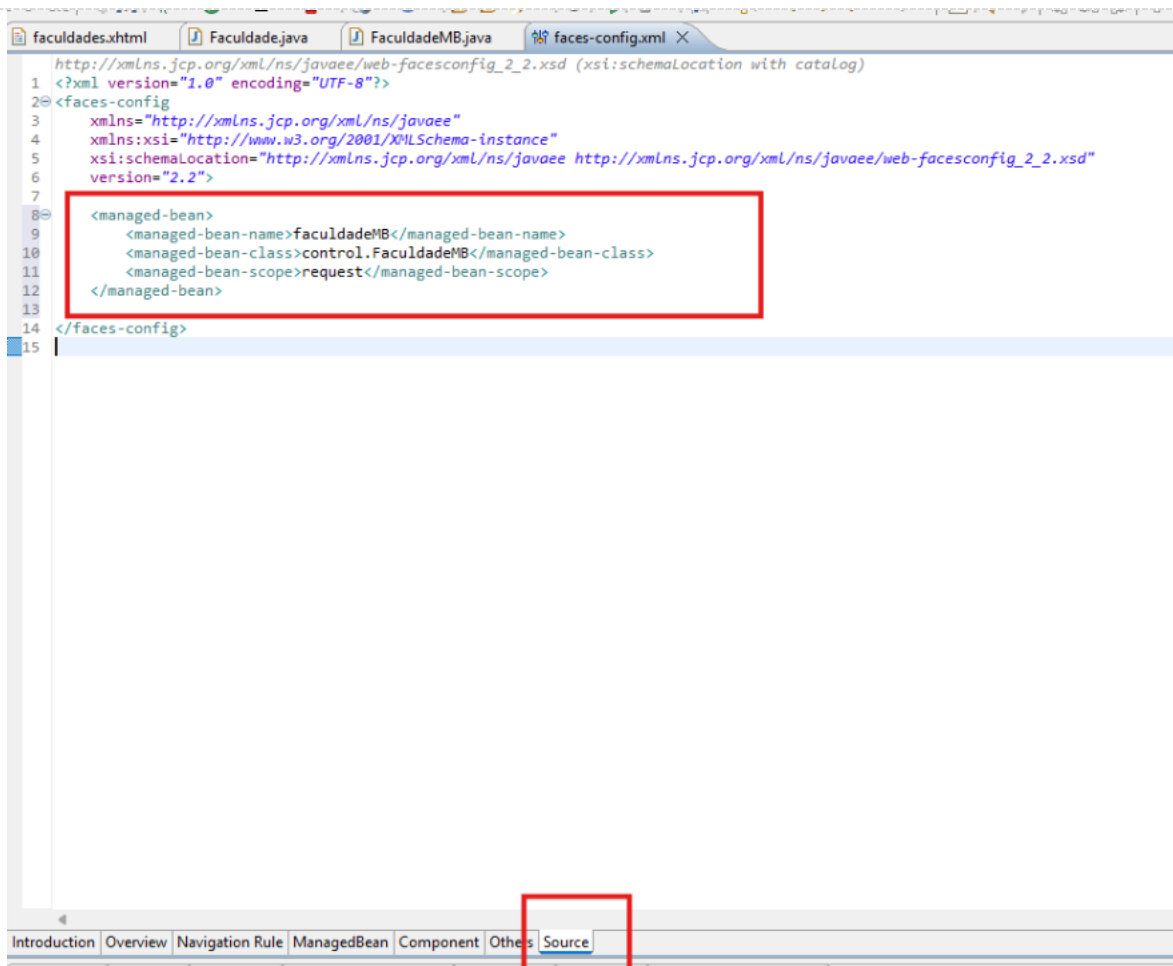


Figura 11 – Classe FaculdadeMB

Para testar, clique com o botão direito no arquivo faculdades.xhtml e selecione Run as > Run on Server. Caso uma janela se abra pedindo para selecionar o server, procure por “WildFly”, selecione “WildFly 10.x” e clique em next (Figura 12), prossiga até finalizar.

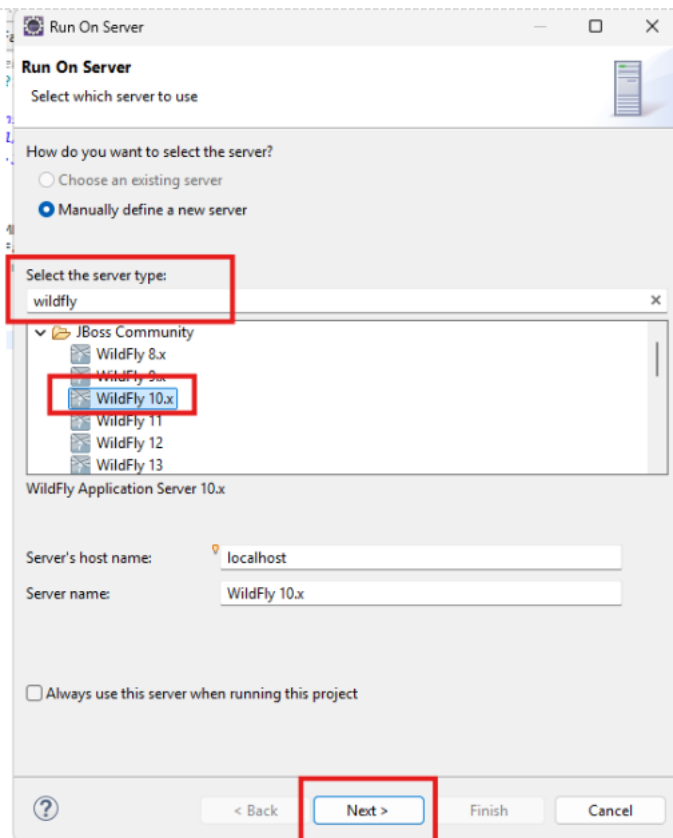


Figura 12 – Selecionar server

Caso de o erro 404 ao rodar o server, siga os seguintes passos:

1. No Eclipse, vá em:
 - a. Project → Properties → Java Compiler
2. Marque “Enable project specific settings”.
3. Em Compiler compliance level, selecione 1.8.
4. Vá em Project → Clean... e limpe/recompile o projeto.
5. Pare e reinicie o WildFly.

Se todos os códigos foram feitos corretamente, o resultado após preencher e clicar no botão submit, deverá ser similar ao da Figura 13.

Código:

Nome completo:

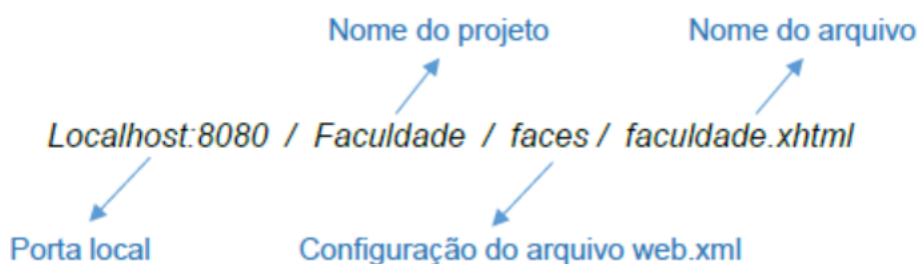
Nome fantasia:

Faculdade cadastrada:
 Código: "100"
 Nome: "Grupo Educacional Super"
 Nome Fantasia: "Super"

Figura 12 – Resultado esperado até o momento

Observe o caminho:

Localhost:8080/Faculdade/faces/faculdade.xhtml



A tag `<url-pattern>` no arquivo `web.xml` é quem especifica o mapeamento do JSF, veja como está para esse projeto. Na linha 13 `/faces/*` implica diretamente no caminho.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" >
3   <display-name>Faculdade</display-name>
4   <servlet>
5     <servlet-name>Faces Servlet</servlet-name>
6     <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
7     <load-on-startup>1</load-on-startup>
8   </servlet>
9   <servlet-mapping>
10    <servlet-name>Faces Servlet</servlet-name>
11    <url-pattern>/faces/*</url-pattern>
12  </servlet-mapping>
13 </web-app>
14
  
```

Agora passaremos para a finalização do projeto criando o arquivo de estilos. Dando continuidade crie uma nova pasta, dentro de "webapp", chamada "resources". Dentro dessa nova pasta, crie uma nova, chamada "css". Quando construímos essa estrutura de pastas, basta adicionar a seguinte diretriz para que o arquivo.xhtml aplique as regras do CSS:

`<h:outputStylesheet library="css" name="arquivo.css" />`

Onde `arquivo.css` é o nome que dará ao arquivo de folha de estilo.

Vamos criar, dentro da pasta `css` o arquivo `estilo1.css` com o seguinte código:

```

1  .rotulo {
2      font-family: Arial, Sans-serif;
3      text-transform: uppercase;
4      font-size: 12pt;
5      font-weight: bold;
6  }
7  .entrada{
8      border: 1px inset;
9      background-color: #D3D3D3;
10     color: #000000;
11     display: block;
12 }
13

```

Pare a execução do servidor, e execute novamente. Veja se o resultado é similar ao da Figura 13.

CÓDIGO:

NOME COMPLETO:

NOME FANTASIA:

Faculdade cadastrada:

Código: "0"

Nome: ""

Nome Fantasia: ""

Figura 13 – Resultado da atividade

Avaliando os resultados:

Ao final da aula, deverá ser entregue um projeto web chamado Faculdade, com uma interface gráfica, uma classe modelo e uma classe de controle. O sistema deverá estar funcionando.

Utilize esta tabela para organizar os passos realizados e adicionar capturas de tela:

Estrutura do Projeto Criada	[Insira o Print]
Arquivo faculdades.xhtml	[Insira o Print]
Arquivo Faculdade.java	[Insira o Print]
Arquivo FaculdadeMB.java	[Insira o Print]
Arquivo estilo1.css	[Insira o Print]
Resultado Testado	[Insira o Print]

Checklist:

- ✓ Estrutura do projeto criada corretamente;
- ✓ Arquivo xhtml criado;
- ✓ Classe modelo para o objeto Faculdade;
- ✓ Arquivos organizados nos devidos pacotes;
- ✓ ManagedBean criado;
- ✓ Configuração do arquivo faces-config.xml
- ✓ Projeto executando em um browser
- ✓ Pasta “resources” criada dentro de “webapp”
- ✓ Pasta “css” criada dentro de “resources”
- ✓ Arquivo css chamado estilo1.css criado dentro da pasta css
- ✓ Arquivo estilo1.css anexado no arquivo xhtml através da referência:
 - ☐ <h:outputStylesheet library="css" name="estilo1.css" />

RESULTADOS

Resultados do experimento:

O arquivo deverá conter:

- *Capa*
- *Folha de rosto com os dados da disciplina e do aluno*
- *Métodos: Explique as técnicas que utilizou*
- Resultado:
 - Captura de tela da página WEB.
 - Captura de tela da estrutura do projeto.
- Evidências (códigos):
 - faculdades.xhtml
 - Faculdade.java
 - FaculdadeMB.java
 - estilo1.css

Resultados de Aprendizagem:

Após concluir esta atividade, você terá aprendido:

- Criar um projeto web dinâmico em Java utilizando o servidor de aplicações WildFly.
- Estruturar uma aplicação seguindo o padrão MVC (Model–View–Controller).
- Desenvolver páginas de interface utilizando JSF (JavaServer Faces) e componentes XHTML.
- Implementar Managed Beans para comunicação entre a camada de visão e a camada de modelo.

- Aplicar estilos personalizados em páginas JSF utilizando folhas de estilo (CSS) e a pasta de recursos padrão do framework.

ROTEIRO DE AULA PRÁTICA

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB

Unidade: U2_DESENVOLVIMENTO_DE_APLICAÇÕES_EM_JSF_COM_PERSISTÊNCIA

Aula: A2_FRAMEWORK_HIBERNATE

OBJETIVOS

Definição dos objetivos da aula prática:

Nessa aula o objetivo é criar um sistema de agenda fazendo a persistência dos dados usando o framework Hibernate.

Para isso o sistema será dividido em duas partes:

- Na primeira deverão ser criados os arquivos de configuração com o banco, bem como a transação de inserção de dados.
- Na segunda, serão criados os arquivos necessários à interação do usuário com o sistema, ou seja, a Managed Bean e o arquivo.xhtml.

SOLUÇÃO DIGITAL:

Eclipse, Wildfly e XAMPP

Ide Eclipse ou correlata que possua configurações para construção de classes seguindo o Java EE, com configurações para ter servidor de aplicação e utilizar o framework JSF para a camada de visão. Recomenda-se o uso do servidor wilfly.

1. Acesse o site oficial: <https://eclipseide.org/>
2. Clique em Download:



3. Escolha a versão “Eclipse IDE for Enterprise Java and Web Developers” que compatível com o sistema operacional do seu computador.



4. Após o download, descompacte o já estará pronto para uso.

PROCEDIMENTOS PRÁTICOS E APLICAÇÕES

Procedimento/Atividade nº 1

DESENVOLVIMENTO DE APLICAÇÃO WEB COM JSF E HIBERNATE – AGENDA DE CONTATOS.

Atividade proposta: Criaremos um projeto baseado em Java EE (Projeto Web Dinâmico), organizado segundo o padrão MVC, para o cadastro de contatos.

Você deve elaborar, pelo menos, uma classe de modelo chamada Contato, contendo os atributos id (Long), nome (String), telefone (String) e email (String). Esses dados serão obtidos por meio de um formulário presente na página Contato.xhtml, na qual o usuário poderá cadastrar, listar, alterar e excluir registros.

A página deve utilizar os componentes e tags do JSF, de forma compatível com as premissas de utilização de ManagedBeans. A classe Java ContatoMB será responsável por controlar as requisições oriundas da camada de visão e encaminhá-las à camada de persistência.

A persistência dos dados deve ser implementada por meio da API JPA, com o provedor Hibernate, realizando a integração com o banco de dados MySQL. A configuração do acesso ao banco será feita no arquivo persistence.xml, e as operações de CRUD serão estruturadas nas classes DAO e CRUD, utilizando EntityManager para gerenciar as transações.

As etapas descritas a seguir foram desenvolvidas utilizando a IDE Eclipse e o servidor WildFly, em conjunto com o XAMPP (phpMyAdmin) para gerenciamento do banco de dados.

Procedimentos para a realização da atividade:

1. Usando o Xampp (ou outro SGBD), crie um banco de dados chamado “agenda”.
2. Faça o download do conector MySql pelo link:
 - a. <https://repo1.maven.org/maven2/mysql/mysql-connector-java/8.0.11/mysql-connector-java-8.0.11.jar>
3. Faça o download do Hibernate, os links para download das libs:
 - a. <https://repo1.maven.org/maven2/antlr/antlr/2.7.7/antlr-2.7.7.jar>
 - b. <https://repo1.maven.org/maven2/com/fasterxml/classmate/1.3.0/classmate-1.3.0.jar>
 - c. <https://repo1.maven.org/maven2/dom4j/dom4j/1.6.1/dom4j-1.6.1.jar>
 - d. <https://repo1.maven.org/maven2/org/hibernate/common/hibernate-commons-annotations/5.0.1.Final/hibernate-commons-annotations-5.0.1.Final.jar>
 - e. <https://repo1.maven.org/maven2/org/hibernate/hibernate-core/5.2.17.Final/hibernate-core-5.2.17.Final.jar>
 - f. <https://repo1.maven.org/maven2/org/hibernate/hibernate-entitymanager/5.0.7.Final/hibernate-entitymanager-5.0.7.Final.jar>
 - g. <https://repo1.maven.org/maven2/org/javassist/javassist/3.18.1-GA/javassist-3.18.1-GA.jar>
 - h. <https://repo1.maven.org/maven2/org/jboss/logging/jboss-logging/3.3.0.Final/jboss-logging-3.3.0.Final.jar>
4. Crie um projeto Dynamic Web com o nome “Agenda” (Figura 1).
5. Dentro da pasta webapp > WEB-INF > lib, coloque tanto o conector MySql, quanto os arquivos do Hibernate que você realizou o download nos passos 2 e 3.
6. Clique com o botão direito no nome do projeto, selecione “Properties” e depois Java Build Path > aba Libraries. Clique em Add Jars e selecione a pasta lib que você copiou os arquivos no passo 6 (Figura 2).
7. Ajustar o compilador

- a. Clique com o botão direito no projeto Agenda → Properties
 - b. Vá em Java Compiler
 - c. Marque *Enable project specific settings*
 - d. Em Compiler compliance level, selecione 1.8
 - e. Clique em Apply and Close
- 8. Ajustar o “Project Facet”
 - a. Clique novamente com o botão direito no projeto → Properties
 - b. Vá em Project Facets
 - c. Procure a linha Java
 - d. Altere a Version para 1.8
 - i. Se estiver cinza:
 - 1. Clique no link “Further configuration available...”
 - 2. No campo “Runtimes”, marque o WildFly 10 Runtime (ou configure se ainda não existir)
 - 3. Clique em OK e volte para 1.8
 - e. Clique em Apply and Close

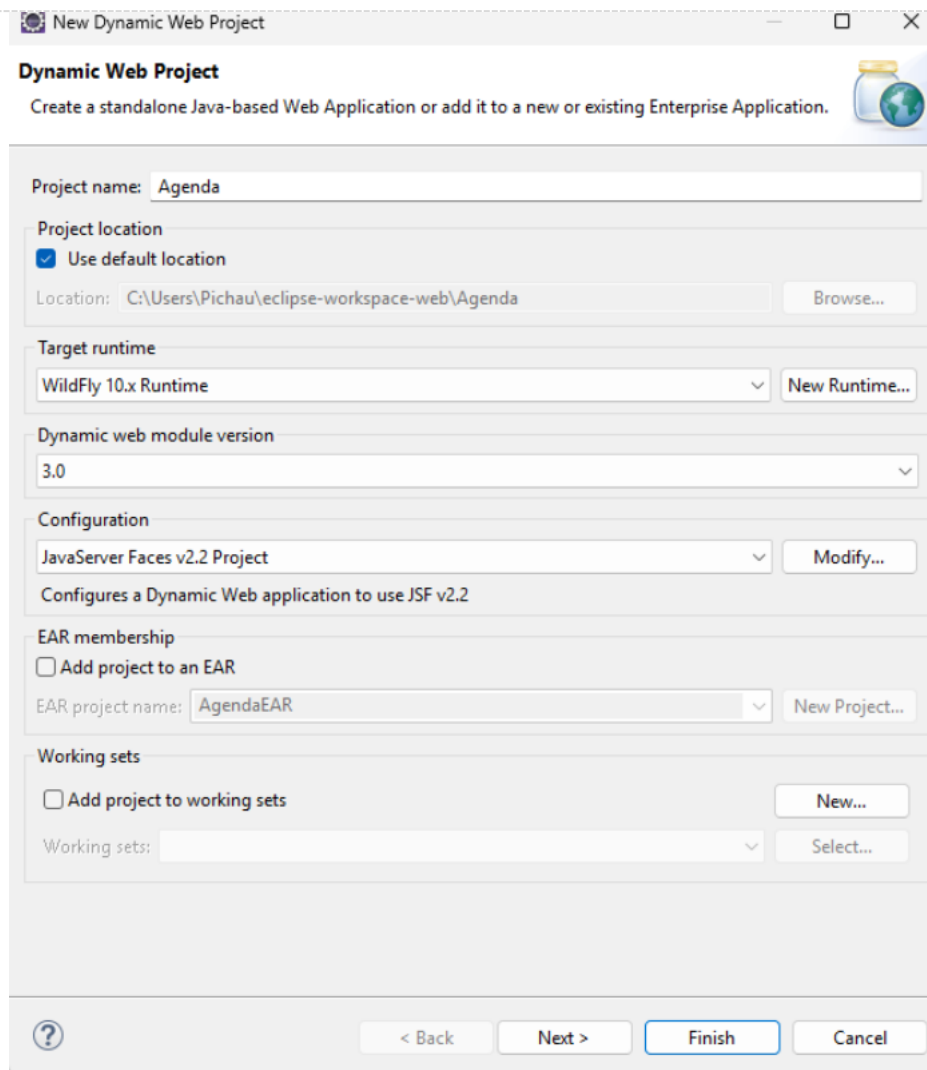


Figura 1 – Criando o projeto

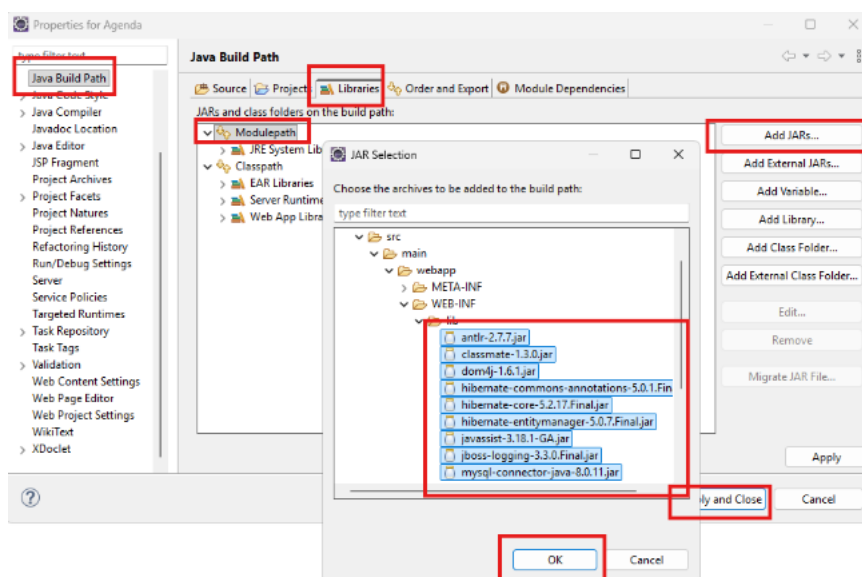


Figura 2 – Adicionando libs

8. Dentro da pasta `webpp> META-INF` crie o arquivo `persistence.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
version="2.1">
<persistence-unit name="agendaPersist" transaction-type="RESOURCE_LOCAL">
<provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

<class>model.Contato</class>

<properties>
<property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/agenda?serverTimezone=UTC"/>
<property name="javax.persistence.jdbc.user" value="root"/>
<property name="javax.persistence.jdbc.password" value=""/>
<property name="javax.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>

<property name="hibernate.show_sql" value="true"/>
<property name="hibernate.format_sql" value="true"/>
<property name="hibernate.dialect" value="org.hibernate.dialect.MySQL5Dialect"/>
<property name="hibernate.hbm2ddl.auto" value="create"/>
</properties>
</persistence-unit>
</persistence>

```

9. Crie um pacote chamado “*model*” dentro da pasta raiz “*src*” e crie a classe “Contato”, com o seguinte código:

```

package model;

import javax.persistence.*;

@Entity
@Table(name = "Contato")
public class Contato {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column
    private int id;
    @Column
    private String nome;
}

```

```
@Column
private String sobrenome;
@Column
private String telefone;
@Column
private String email;

public int getId() {
return id;
}

public void setId(int id) {
this.id = id;
}

public String getNome() {
return nome;
}

public void setNome(String nome) {
this.nome = nome;
}

public String getSobrenome() {
return sobrenome;
}

public void setSobrenome(String sobrenome) {
this.sobrenome = sobrenome;
}

public String getTelefone() {
return telefone;
}

public void setTelefone(String telefone) {
this.telefone = telefone;
}

public String getEmail() {
```

```
return email;
}
```

```
public void setEmail(String email) {
    this.email = email;
}
}
```

10. Dentro da pasta model crie a classe DAO, para fazer a conexão com o banco

```
package model;
```

```
import javax.persistence.*;
```

```
public class DAO {
    private static final EntityManagerFactory emFactory;
```

```
static {
    emFactory = Persistence.createEntityManagerFactory("agendaPersist");
}
```

```
public static EntityManager getEntityManager() {
    return emFactory.createEntityManager();
}
```

```
public static void fecharFactory() {
    emFactory.close();
}
}
```

11. Dentro da pasta model crie agora a classe CRUD.

```
package model;
```

```
import javax.persistence.*;
```

```
public class CRUD {
    public static void inserir(Contato c1) {
        try {
            EntityManager entityManager = DAO.getEntityManager();
```



```

entityManager.getTransaction().begin();
entityManager.persist(c1);
entityManager.getTransaction().commit();
entityManager.close();
System.out.println("conectado Salvo!");
} catch (Exception ex) {
ex.printStackTrace();
}
}
}

```

Nesse momento, fique à vontade caso queira implementar as demais transações com o banco, elas estão disponíveis no texto principal da seção 2.3.

Agora vamos criar os componentes necessários para a interação do usuário com o sistema.

1. Crie o pacote “*controller*” dentro da pasta raiz “*src*” e dentro dele crie a classe ContatoMB com o seguinte código:

```

package controller;

import javax.faces.bean.ManagedBean;
import model.*;

@ManagedBean
public class ContatoMB {
    public ContatoMB() {

    }

    private Contato c = new Contato();

    public String getId() {
        return String.valueOf(c.getId());
    }

    public void setId(String id) {
        c.setId(Integer.parseInt(id));
    }

    public String getNome() {
        return c.getNome();
    }
}

```

```

public void setNome(String nome) {
    c.setNome(nome);
}

public String getSobrenome() {
    return c.getSobrenome();
}

public void setSobrenome(String sobrenome) {
    c.setSobrenome(sobrenome);
}

public String getTelefone() {
    return c.getTelefone();
}

public void setTelefone(String telefone) {
    c.setTelefone(telefone);
}

public void setEmail(String email) {
    c.setEmail(email);
}

public String getEmail() {
    return c.getEmail();
}

public void salvar() {
    try {
        CRUD.inserir(c);
    } catch (Exception ex){
        ex.printStackTrace();
    }
}
}

```

2. Configure arquivo faces-config.xml na pasta *webapp*>WEB-INF, deixando com o seguinte código:

```

<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
version="2.2">
```

```
<managed-bean>
<managed-bean-name>contatoMB</managed-bean-name>
<managed-bean-class>controller.ContatoMB</managed-bean-class>
<managed-bean-scope>request</managed-bean-scope>
</managed-bean>

</faces-config>
```

3. Por fim, crie o arquivo Contato.xhtml dentro da pasta webapp, com o seguinte código:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:h="http://xmlns.jcp.org/jsf/html">
<h:head>
<meta charset="UTF-8" />
<title>Cadastro de Contatos</title>
</h:head>
<h:body>
<h2>Novo Contato</h2>
<h:form>
<p>Nome:
<h:inputText value="#{contatoMB.nome}"/>
</p>
<p>Sobrenome:
<h:inputText value="#{contatoMB.sobrenome}"/>
</p>
<p>Telefone:
<h:inputText value="#{contatoMB.telefone}"/>
</p>
<p>Email:
<h:inputText value="#{contatoMB.email}"/>
</p>

<h:commandButton value="Salvar" action="#{contatoMB.salvar}" />
</h:form>
<p>Cadastro confirmado:
Código: #{contatoMB.id}
Nome: #{contatoMB.nome} #{contatoMB.sobrenome}
```

```
</p>  
</h:body>  
  
</html>
```

Com a criação dos 6 arquivos de códigos descritos nas duas atividades, juntamente com as configurações que foram realizadas, o projeto “Agenda” está pronto para ser executado e persistir dados.

Checklist:

- ✓ Criar o Banco de dados Usuários no MySQL;
- ✓ Criação da estrutura do projeto no Eclipse;
- ✓ Criação o arquivo de persistência;
- ✓ Criação da classe Contato.java;
- ✓ Criação da classe DAO.java;
- ✓ Criação da classe CRUD.java;
- ✓ Criação da classe *Managed Bean* chamada “ContatoMB”;
- ✓ Configuração do arquivo faces-config.xml;
- ✓ Criação da interface gráfica em um arquivo xhtml.

RESULTADOS

Resultados do experimento:

O arquivo deverá conter:

- *Capa*
- *Folha de rosto com os dados da disciplina e do aluno*
- *Métodos: Explique as técnicas que utilizou*
- Resultado:
 - Captura de tela da raiz do projeto;
 - Captura de tela da página Contato funcionando;
 - Captura de tela da tabela do banco de dados com pelo menos 5 dados inseridos.
- Evidências (códigos):
 - Persistence.xml
 - Contato.java
 - DAO.java
 - CRUD.java
 - ContatoMB.java

o Contato.xhtml

Resultados de Aprendizagem:

Após concluir esta atividade, você terá aprendido:

- Criação e configuração de um projeto Java EE (Web Dinâmico) estruturado segundo o padrão MVC.
- Implementação de um CRUD (Create, Read, Update, Delete) utilizando JSF como camada de visão.
- Mapeamento de entidades e persistência de dados com JPA/Hibernate integrando com o banco de dados MySQL.
- Configuração e utilização do arquivo persistence.xml para definir o provedor de persistência e os parâmetros de conexão.
- Manipulação de dados por meio da API EntityManager, aplicando boas práticas de transações e controle de exceções.
- Integração entre as camadas View, Controller e Model, garantindo comunicação eficiente entre os componentes do sistema.

ROTEIRO DE AULA PRÁTICA

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB II

Unidade: U3_DESENVOLVIMENTO_DE_APLICAÇÕES_EM_JSF_E_SUAS_EXTENSÕES

Aula: A3_FRAMEWORK_TOMAHAWK_HIGHFACES_E_BOOFACES

OBJETIVOS

Definição dos objetivos da aula prática:

- Compreender o papel dos frameworks Tomahawk, HighFaces e BootsFaces no ecossistema JSF, reconhecendo como eles ampliam os recursos visuais e funcionais das aplicações web.
- Explorar a integração do BootsFaces com o JSF, aplicando componentes visuais baseados em Bootstrap para criar interfaces modernas, responsivas e de fácil manutenção.
- Desenvolver uma aplicação prática utilizando BootsFaces, aplicando seus principais componentes (como `<b:form>`, `<b:inputText>`, `<b:commandButton>`, `<b:dataTable>`, entre outros) em conjunto com ManagedBeans e a arquitetura MVC.

SOLUÇÃO DIGITAL:

Eclipse e BootsFaces

Ide Eclipse ou correlata que possua configurações para construção de classes seguindo o Java EE, com configurações para ter servidor de aplicação e utilizar o framework JSF para a camada de visão. Recomenda-se o uso do servidor wilfly.

1. Acesse o site oficial: <https://eclipseide.org/>
2. Clique em Download:



- Escolha a versão “Eclipse IDE for Enterprise Java and Web Developers” que compatível com o sistema operacional do seu computador.



- Após o download, descompacte o já estará pronto para uso.

PROCEDIMENTOS PRÁTICOS E APLICAÇÕES

Procedimento/Atividade nº 1

DESENVOLVIMENTO DE INTERFACES JSF COM BOOTSFACES E DATATABLE

Atividade proposta:

Criaremos um projeto baseado em Java EE (Projeto Web Dinâmico) utilizando o framework JSF e a biblioteca BootsFaces, com o objetivo de desenvolver uma interface moderna e responsiva para listagem de veículos.

Você deverá elaborar uma classe de modelo chamada Veículo, contendo os atributos id (Integer), modelo (String), fabricante (String), ano (int) e valor (double). Esses dados serão manipulados por meio de uma tabela dinâmica na página veiculos.xhtml, utilizando o componente `<b:dataTable>` do BootsFaces.

O projeto deve seguir o padrão MVC, utilizando uma classe ManagedBean (VeiculoMB) para controlar as ações do usuário e repassar os dados para a camada de modelo. A tabela deve permitir a exibição dos registros cadastrados e aplicar o estilo visual oferecido pelo BootsFaces, garantindo uma interface limpa e intuitiva.

As etapas descritas a seguir foram desenvolvidas na IDE Eclipse, com o servidor WildFly, demonstrando a integração entre JSF e BootsFaces na criação de aplicações web modernas e compatíveis com o padrão Java EE.

Procedimentos para a realização da atividade:

1. Crie um novo projeto chamado PraticaBootsFaces, com as configurações da Figura 1.

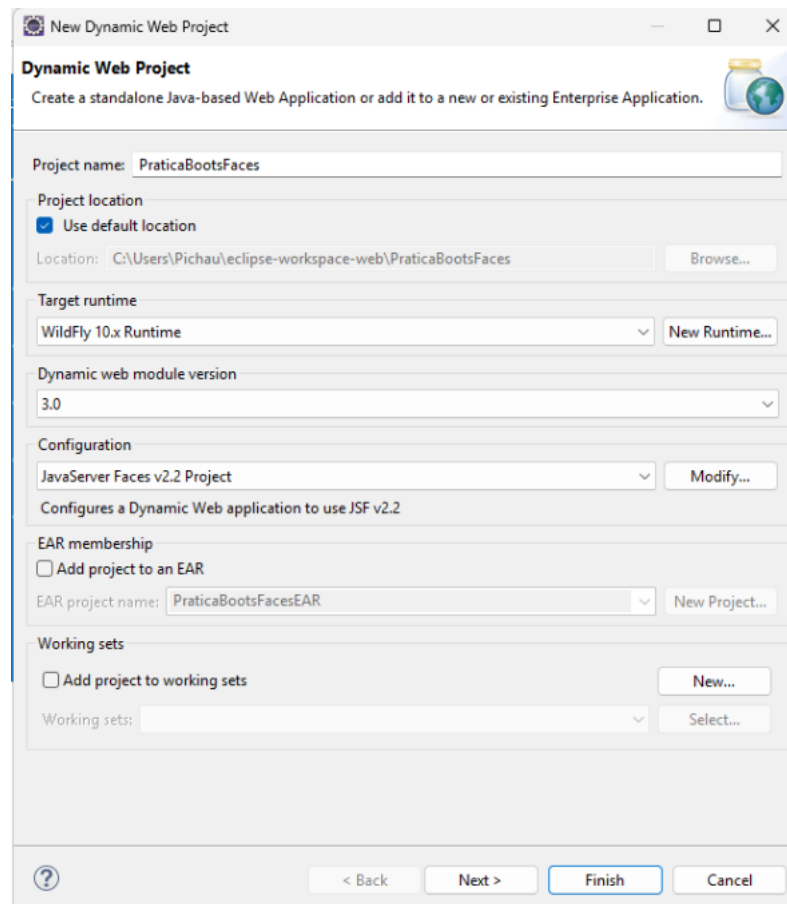


Figura 1 – Criando o projeto

2. Ajustar o compilador

- o Clique com o botão direito no projeto Agenda → Properties
- o Vá em Java Compiler
- o Marque Enable project specific settings
- o Em Compiler compliance level, selecione 1.8
- o Clique em Apply and Close

3. Dentro da pasta java raiz “src”, criar um novo pacote chamado model. Dentro desse pacote criar a classe Carro.java com o seguinte código:

```
package model;

public class Carro {

    private String modelo;
    private String marca;

    public Carro(String marca, String modelo) {
        this.modelo = modelo;
        this.marca = marca;
    }

    public String getModelo() {
        return modelo;
    }
}
```



```
public void setModelo(String modelo) {
    this.modelo = modelo;
}
```

```
public String getMarca() {
    return marca;
}
```

```
public void setMarca(String marca) {
    this.marca = marca;
}
}
```

4. Dentro da pasta java raiz “src”, criar um novo pacote chamado control, e dentro dele criar uma nova classe java chamada CarrosMB.java, com o seguinte código:

```
package control;
```

```
import java.util.ArrayList;
import java.util.List;
import javax.faces.bean.ManagedBean;
import model.Carro;
```

```
@ManagedBean
public class CarrosMB {

    public CarrosMB() {
        carros = new ArrayList<Carro>();
        carros.add(new Carro("Marca 1", "Modelo 1"));
        carros.add(new Carro("Marca 2", "Modelo 2"));
        carros.add(new Carro("Marca 3", "Modelo 3"));
    }
}
```

```
private List<Carro> carros;
```

```
public List<Carro> getCarros() {
    return carros;
}
```

```
public void setCarros(List<Carro> carros) {
    this.carros = carros;
}
}
```

5. Dentro do arquivo faces-config.xml acrescentar a seguinte configuração:

```
<managed-bean>
<managed-bean-name>carrosMB</managed-bean-name>
<managed-bean-class>control.CarrosMB</managed-bean-class>
<managed-bean-scope>request</managed-bean-scope>
</managed-bean>
```

Agora vamos criar a interface gráfica para usar os dados da Managed Bean CarrosMB.java.

Acesse o site [https://github.com/TheCoder4eu/BootsFaces-](https://github.com/TheCoder4eu/BootsFaces-OSP/releases/download/v1.3.0/BootsFaces-OSP-1.3.0-dist.jar)

OSP/releases/download/v1.3.0/BootsFaces-OSP-1.3.0-dist.jar e faça o download do framework BootsFaces. Descompacte o arquivo e copie o JAR para a pasta lib do projeto. Utilizamos o arquivo “BootsFaces-OSP-1.3.0-dist”.

Dentro da pasta webapp crie um arquivo chamado pagina.xhtml, com o respectivo código:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:h="http://java.sun.com/jsf/html"
xmlns:f="http://java.sun.com/jsf/core"
xmlns:b="http://bootsfaces.net/ui">
<h:head>
<title>Aula prática</title>
</h:head>
<h:body>
<h:form>
<b:container fluid="true">
<b:jumbotron>
<h2>KLS 2.0</h2>
<p>Programação para web 2 - JSF</p>
<p> <a href="link" class="btn btn-lg btn-info">Veja mais. </a> </p>
</b:jumbotron>

<b:panel title="Painel 1">
<h2>Veículos disponíveis</h2>
<b:dataTable value="#{carrosMB.carros}" var="car">
<b:dataTableColumn value="#{car.marca}" />
<b:dataTableColumn value="#{car.modelo}" />
</b:dataTable>
</b:panel>
<b:panel title="Painel 2">
<b:inputText placeholder="Digite seu nome completo" label="Nome:" />
<b:datepicker label="Data de nascimento" />
<b:selectOneMenu label="Possui habilitação">
<f:selectItem itemValue="sim" itemLabel="sim" />
<f:selectItem itemValue="nao" itemLabel="não" />
</b:selectOneMenu>
<b:selectMultiMenu label="Selecione vários">
<f:selectItem itemValue="op1" itemLabel="Habilitação para moto" />
<f:selectItem itemValue="op2" itemLabel="Habilitação para carro" />
<f:selectItem itemValue="op2" itemLabel="Habilitação para caminhão" />
</b:selectMultiMenu>
</b:panel>
</b:container>
</h:form>
</h:body>

</html>
```

Se todos os passos foram devidamente seguidos, o resultado será como na Figura 2.

KLS 2.0

Programação para web 2 - JSF

[Veja mais.](#)

Panel 1

Veículos disponíveis

Show 10 entries

Search:

Marca	Modelo
Marca 1	Modelo 1
Marca 2	Modelo 2
Marca 3	Modelo 3

Showing 1 to 3 of 3 entries

Previous [1](#) Next

Panel 2

Nome:

Data de nascimento

Business Institution

Figura 2 – Resultado do roteiro

Avaliando os resultados:

Após o experimento responda as seguintes questões:

- Qual é a principal vantagem de usar o BootsFaces em relação aos componentes padrão do JSF na criação das interfaces web?
- Como o uso de componentes BootsFaces, como `<b:inputText>` e `<b:dataTable>`, contribui para a usabilidade e organização visual da aplicação?
- De que forma o BootsFaces se integra ao JSF, e por que essa integração é importante para o funcionamento da aplicação?

Checklist:

- ✓ Criar um projeto com as especificações sugeridas
- ✓ Criar o pacote model e dentro dele a classe Carro.java
- ✓ Criar o pacote model e dentro dele a classe CarrosMB.java
- ✓ Acrescentar a configuração da Managed Bean no arquivo faces-config.xml
- ✓ Fazer o download do framework BootsFaces
- ✓ Copiar o JAR para a pasta lib do projeto
- ✓ Dentro da pasta webappcriar o arquivo pagina.xhtml com o devido código.
- ✓ Responder as questões.

RESULTADOS

Resultados do experimento:

O arquivo deverá conter:

- *Capa*
- *Folha de rosto com os dados da disciplina e do aluno*
- *Métodos: Explique as técnicas que utilizou*
- Resultado:
 - Respostas das questões do avaliando resultados.
- Evidências:
 - Captura de tela do projeto.
 - Captura de tela do resultado do experimento.

Formato de entrega:

- Arquivo PDF

Resultados de Aprendizagem:

Após concluir esta atividade, você terá aprendido:

- A compreender a função dos frameworks Tomahawk, HighFaces e BootsFaces no aprimoramento das aplicações desenvolvidas com JSF.
- A utilizar componentes visuais do BootsFaces para construir interfaces web modernas, responsivas e integradas ao padrão MVC.
- A integrar o BootsFaces ao JSF, configurando corretamente as bibliotecas e aplicando seus recursos de forma prática no desenvolvimento de páginas XHTML.
- A diferenciar os frameworks visuais para JSF, reconhecendo suas vantagens e o contexto mais adequado para o uso de cada um.

ROTEIRO DE AULA PRÁTICA

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB II

Unidade: U4_DESENVOLVIMENTO_DE_APLICAÇÕES_COM_API

Aula: A1_API_RESTFULL

OBJETIVOS

Definição dos objetivos da aula prática:

- Compreender o funcionamento de APIs RESTful e seus princípios básicos.
- Implementar endpoints usando JAX-RS no Java EE.
- Produzir e consumir dados no formato JSON.
- Executar e testar serviços REST em um servidor de aplicação (WildFly).

SOLUÇÃO DIGITAL:

Eclipse

Ide Eclipse ou correlata que possua configurações para construção de classes seguindo o Java EE, com configurações para ter servidor de aplicação e utilizar o framework JSF para a camada de visão. Recomenda-se o uso do servidor wilfly.

1. Acesse o site oficial: <https://eclipseide.org/>
2. Clique em Download:



3. Escolha a versão “Eclipse IDE for Enterprise Java and Web Developers” que compatível com o sistema operacional do seu computador.

Eclipse IDE for Enterprise Java and Web Developers

534 MB | 232,110 DOWNLOADS



Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, Yaml, Markdown, Web Services, JPA and Data Tools, Maven and Gradle, Git, and more.

[Click here to raise an issue with the Eclipse Web Tools Platform.](#)
[Maintainers will move opened issues to the right place.](#)
[Click here to raise an issue with the Eclipse Platform.](#)
[Click here to raise an issue with Maven integration for web projects.](#)
[Click here to raise an issue with Eclipse Wild Web Developer \(incubating\).](#)



Windows: AArch64 | x86_64
macOS: AArch64 | x86_64
Linux: AArch64 | riscv64 | x86_64

4. Após o download, descompacte o já estará pronto para uso.

PROCEDIMENTOS PRÁTICOS E APLICAÇÕES

Procedimento/Atividade nº 1

RESTBOOK – CATÁLOGO DE LIVROS VIA API REST

Atividade proposta:

Nesta atividade, o aluno desenvolverá uma API RESTful utilizando os recursos da especificação JAX-RS presentes no Java EE (através do servidor WildFly).

A aplicação, chamada RestBook, tem como objetivo expor endpoints REST para listar e cadastrar livros em formato JSON, aplicando os princípios de comunicação sem estado (stateless) e padronização de verbos HTTP.

A classe Livro representará o modelo principal da aplicação, contendo os atributos id, título e autor.

Em seguida, será criada a classe LivroResource, responsável por definir os endpoints com as anotações @Path, @GET, @POST e @Produces(MediaType.APPLICATION_JSON).

Esses endpoints permitirão consultar todos os livros e cadastrar novos livros via requisições HTTP.

Procedimentos para a realização da atividade:

1) Criar o projeto (Figura 1)

- a. No Eclipse: File > New > Dynamic Web Project.
- b. Nome: RestBook.
- c. Target runtime: selecione o WildFly instalado.
- d. Configuration: "Default Configuration for WildFly".
- e. Finish.

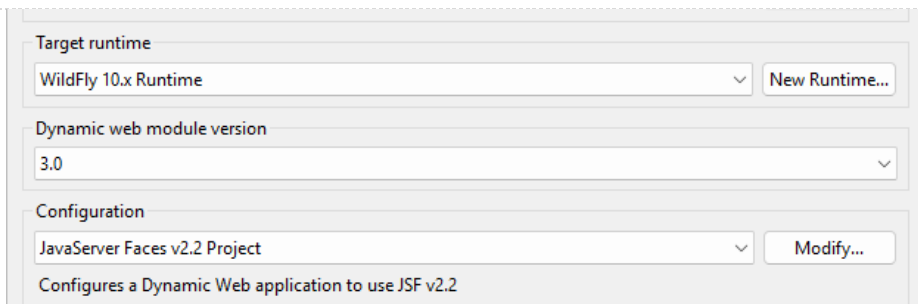


Figura 1 – Criando o projeto

2) **Certifique-se de que o projeto está com Java 8**

- a. Project > Properties > Project Facets: Java 1.8

3) **Habilitar JAX-RS (RESTEasy) e JSON**

O WildFly já traz o RESTEasy. Para garantir a serialização em JSON via Jackson:

- a. Crie a pasta: src/main/webapp/WEB-INF/ (se não existir).
b. Dentro dela, crie o arquivo jboss-deployment-structure.xml com o conteúdo:

```
<jboss-deployment-structure>
<deployment>
<dependencies>
<module name="org.jboss.resteasy.resteasy-jackson-provider" services="import"/>
</dependencies>
</deployment>
</jboss-deployment-structure>
```

Isso adiciona o provider Jackson2 do WildFly ao seu WAR (sem precisar baixar libs).

4) **Definir o pacote base**

- a. Crie um pacote Java, por exemplo: br.com.restbook

5) **Configurar o ponto de entrada do JAX-RS**

- a. Crie a classe JaxRsConfig:

```
package br.com.restbook;

import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api")
public class JaxRsConfig extends Application {
// sem código: a anotação já registra o JAX-RS em /api
}
```

A sua API responderá em URLs que começam com /api.

6) **Criar o modelo (Resource Representation)**

- a. Crie a classe Livro:

```
package br.com.restbook;

public class Livro {
private Integer id;
private String titulo;
private String autor;

public Livro() {}
public Livro(Integer id, String titulo, String autor) {
this.id = id;
this.titulo = titulo;
this.autor = autor;
}
```

```

}

public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }
public String getTitulo() { return titulo; }
public void setTitulo(String titulo) { this.titulo = titulo; }
public String getAutor() { return autor; }
public void setAutor(String autor) { this.autor = autor; }
}

package br.com.restbook;

public class Livro {
    private Integer id;
    private String titulo;
    private String autor;

    public Livro() {}
    public Livro(Integer id, String titulo, String autor) {
        this.id = id;
        this.titulo = titulo;
        this.autor = autor;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getAutor() { return autor; }
    public void setAutor(String autor) { this.autor = autor; }
}

```

7) Criar o recurso REST (endpoints)

a. Crie a classe LivroResource:

```

package br.com.restbook;

import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/livros")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class LivroResource {

    private static final List<Livro> LIVROS = new ArrayList<>();
    private static final AtomicInteger SEQ = new AtomicInteger(1);

    static {
        LIVROS.add(new Livro(SEQ.getAndIncrement(), "Clean Code", "Robert C. Martin"));
        LIVROS.add(new Livro(SEQ.getAndIncrement(), "Effective Java", "Joshua Bloch"));
    }

    @GET
    public List<Livro> listar() {
        return LIVROS; // 200 + JSON
    }
}

```



```

@GET
@Path("/{id}")
public Response obter(@PathParam("id") int id) {
    return LIVROS.stream()
        .filter(l -> l.getId() == id)
        .findFirst()
        .<Response>map(l -> Response.ok(l).build())
        .orElse(Response.status(Status.NOT_FOUND).build());
}

@POST
public Response adicionar(Livro novo, @Context UriInfo uriInfo) {
    if (novo == null || novo.getTitulo() == null || novo.getAutor() == null) {
        return Response.status(Status.BAD_REQUEST).entity("{\"erro\":\"Titulo e autor são obrigatórios\"}").build();
    }
    novo.setId(SEQ.getAndIncrement());
    LIVROS.add(novo);

    UriBuilder uriBuilder =
        uriInfo.getAbsolutePathBuilder().path(String.valueOf(novo.getId()));
    return Response.created(uriBuilder.build()).entity(novo).build(); // 201 + Location + JSON
}

@DELETE
@Path("/{id}")
public Response remover(@PathParam("id") int id) {
    boolean removed = LIVROS.removeIf(l -> l.getId() == id);
    return removed ? Response.noContent().build() :
        Response.status(Status.NOT_FOUND).build();
}

```

8) Deploy no WildFly

- a. Run As > Run on Server (selecione o WildFly).
- b. Verifique no Console se a aplicação subiu sem erros.
- c. Acesse no navegador para testar o GET:
 - i. <http://localhost:8080/RestBook/api/livros>

9) Testes rápidos (Postman ou cURL)

- a. Listar livros (GET)
 - i. URL: GET <http://localhost:8080/RestBook/api/livros>
 - ii. Headers: Accept: application/json
- b. Obter por id (GET)
 - i. URL: GET <http://localhost:8080/RestBook/api/livros/1>
- c. Adicionar livro (POST)
 - i. URL: POST <http://localhost:8080/RestBook/api/livros>
 - ii. Headers:
 1. Content-Type: application/json
 2. Accept: application/json
- d. Body (raw/JSON):

```

{
  "titulo": "Domain-Driven Design",
  "autor": "Eric Evans"
}

```

```
}
```

10) Remover (DELETE)

- URL: DELETE <http://localhost:8080/RestBook/api/livros/2>
- cURL exemplo (POST):

```
curl -i -X POST "http://localhost:8080/RestBook/api/livros" \  
-H "Content-Type: application/json" \  
-d '{"titulo":"Refactoring","autor":"Martin Fowler"}'
```

Avaliando os resultados:

Após o experimento responda as seguintes questões:

- Qual é a principal diferença entre um método REST anotado com @GET e outro com @POST?
- Por que o formato JSON é amplamente utilizado em APIs RESTful modernas?
- Explique o papel da anotação @Path em uma classe que representa um recurso REST.

Checklist:

- ✓ Criar o projeto “RestBook” no Eclipse (Dynamic Web Project com suporte a JAX-RS).
- ✓ Configurar a classe Livro com os atributos e métodos getters/setters.
- ✓ Criar a classe LivroResource com os métodos listarLivros() e adicionarLivro().
- ✓ Garantir o retorno em JSON usando @Produces(MediaType.APPLICATION_JSON).
- ✓ Testar os endpoints /livros e /livros/add no Postman ou navegador.

RESULTADOS

Resultados do experimento:

O arquivo deverá conter:

- *Capa*
- *Folha de rosto com os dados da disciplina e do aluno*
- *Métodos: Explique as técnicas que utilizou*
- Resultado:
 - Respostas das questões do avaliando resultados.
- Evidências:
 - Captura de tela do projeto;
 - Captura de tela da página dos dois GETs no navegador.

Formato de entrega:

- Arquivo em Word ou PDF com as capturas de tela.

Resultados de Aprendizagem:

Ao final desta atividade, o aluno será capaz de:

- Compreender a estrutura e funcionamento de uma API RESTful desenvolvida em Java EE, reconhecendo o papel dos recursos, métodos HTTP e formato de troca de dados (JSON).
- Implementar e expor endpoints REST utilizando JAX-RS (RESTEasy), aplicando anotações como `@Path`, `@GET`, `@POST` e `@Produces` para definir o comportamento da API.
- Testar e validar requisições HTTP (GET, POST e DELETE) através de ferramentas como Postman ou navegador, analisando os códigos de status e as respostas retornadas em JSON.
- Configurar e executar uma aplicação REST no servidor WildFly, garantindo o correto funcionamento do módulo de serialização JSON (Jackson Provider).